Kubeflow

Prediction of employee attrition

Ananth M - 1RV22AI007

Gnyan - 1RV22AI017

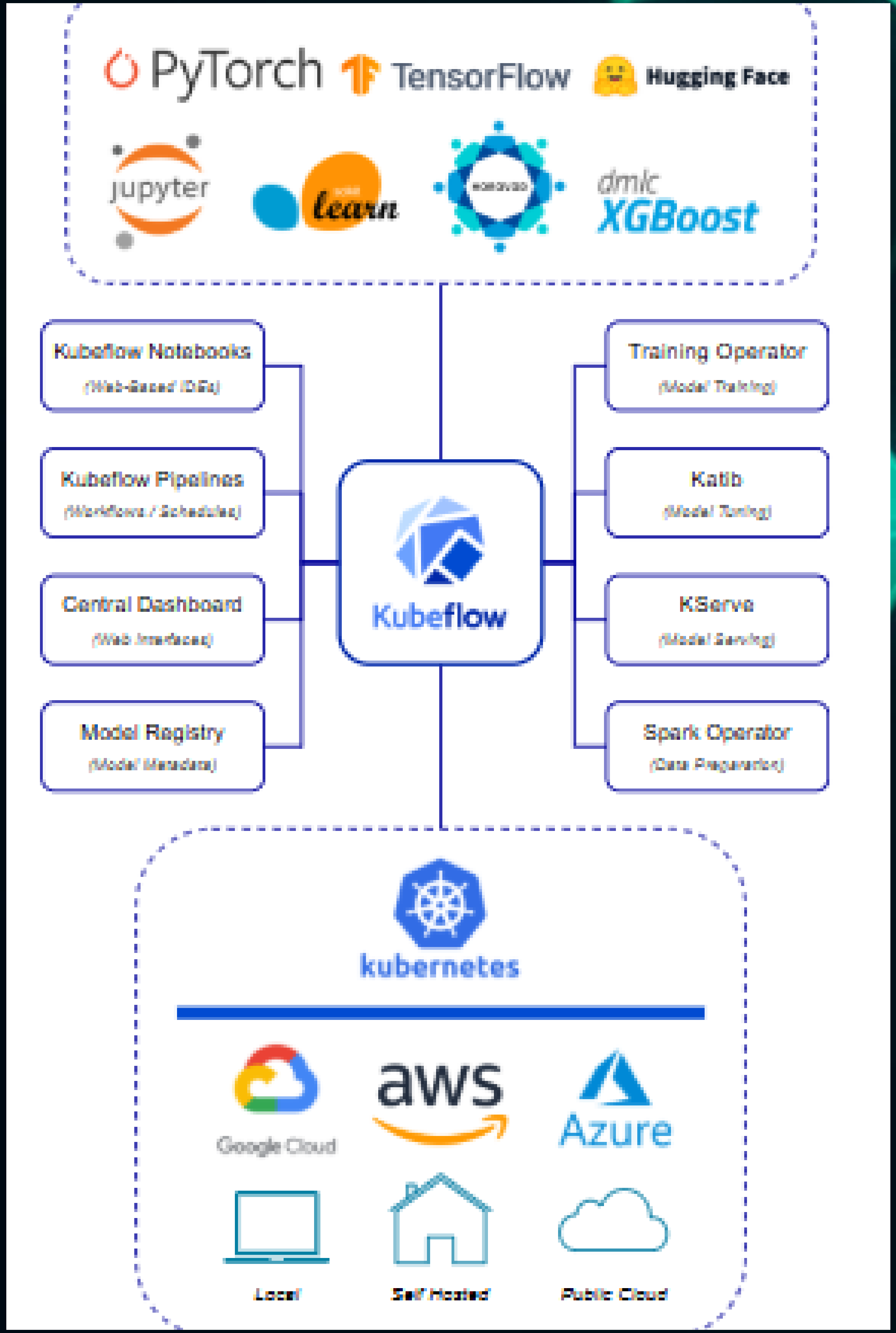
Parth - 1RV22AI037

RajLaxmi - 1RV22AI041

Introduction

- Kubeflow is an open-source platform designed to simplify the development, deployment, and management of machine learning (ML) workflows. It provides tools for building end-to-end ML pipelines, including model training, serving, and monitoring.

- Developed by Google, Kubeflow leverages Kubernetes to deliver scalable and portable infrastructure, allowing data scientists and engineers to collaborate efficiently on ML projects, ranging from predictive analytics to advanced deep learning.



Introduction (Key Points)

- Build and manage end-to-end machine learning workflows :

  All seamlessly integrated with Kubernetes.

- Made at Google :

  For simplifying and scaling machine learning pipelines.

- Made with data scientists and ML engineers in mind :

  To let teams focus more on innovation and less on managing infrastructure.

- High-level tools for defining, deploying, and monitoring ML workflows :

  Making it easy to manage reproducible and scalable pipelines.

- Supports advanced features like hyperparameter tuning and distributed training :

  Includes tools for serving, versioning, and monitoring machine learning models.

Why KubeFlow

Automation

- Reduces manual effort by automating repetitive tasks and workflow execution.

Scalability

- Dynamically handles workloads of any size, leveraging Kubernetes for efficient resource management.

Collaboration

- Modular design allows teams to work on components independently and reuse them across workflows.

Reproducibility

- Guarantees consistent results by encapsulating steps in containers and tracking metadata.

Experimentation

- Simplifies hyperparameter tuning, experiment tracking, and result comparison for rapid iteration.

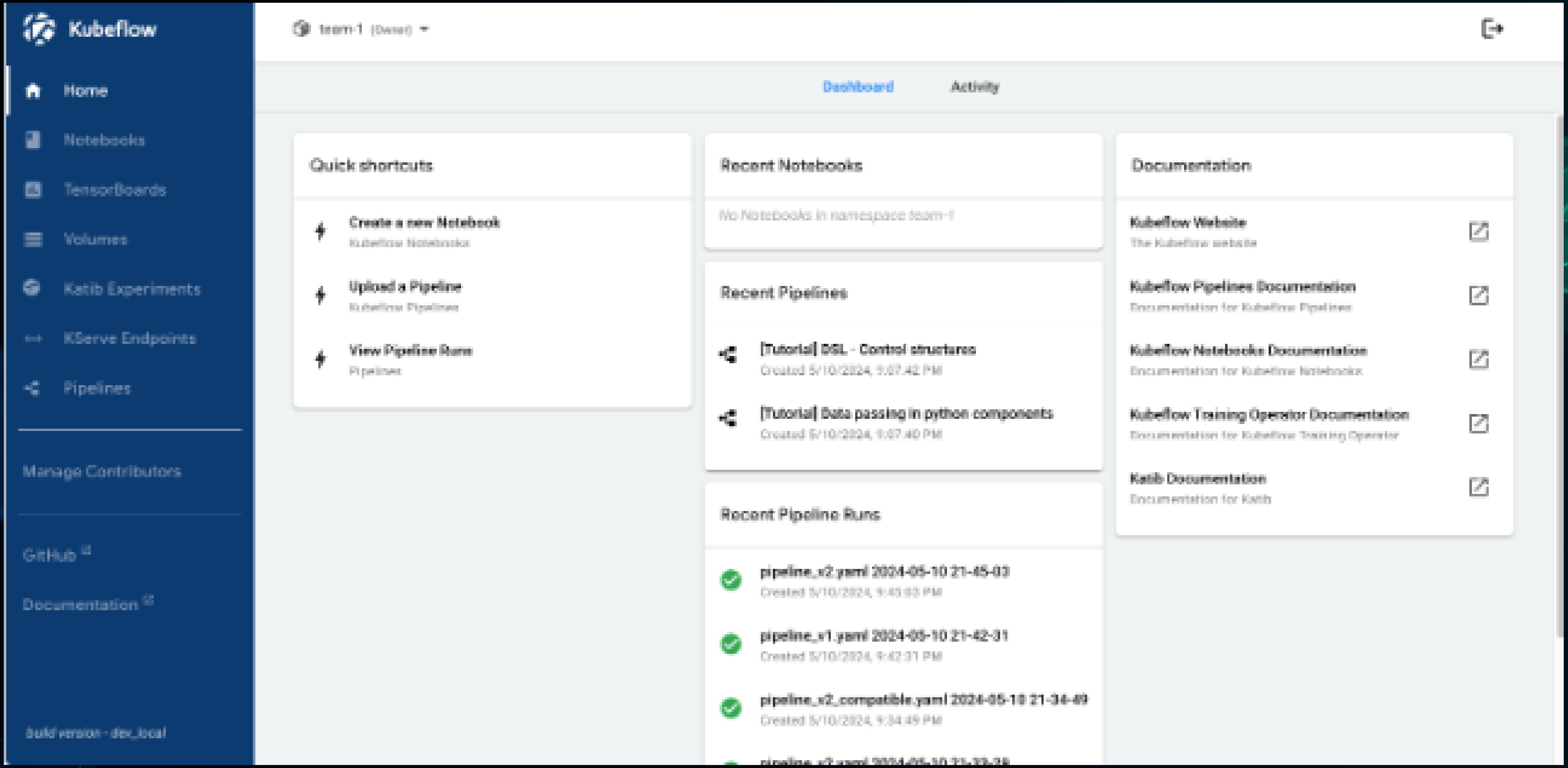
Features Of KubeFlow

1. Orchestration and Workflow Management
   - Built-in tools for orchestrating and managing end-to-end machine learning pipelines.
   - Ensures pipeline steps are executed in sequence and on schedule.
2. Scalability and Portability
   - Leverages Kubernetes for scalable and distributed training and inference.
   - Runs seamlessly across on-premises, cloud, or hybrid environments.
3. Support for ML Frameworks
   - Compatible with popular frameworks like TensorFlow, PyTorch, Scikit-learn, and XGBoost.
   - Provides flexibility to use the right tool for the task.
4. Reproducibility and Versioning
   - Tracks pipeline versions, experiments, and artifacts automatically.
   - Ensures results are reproducible across different environments.

Features Of KubeFlow

5. Hyperparameter Tuning
   - Includes Katib, a tool for automating hyperparameter optimization.
   - Supports multiple algorithms like grid search, random search, and Bayesian optimization.
6. Model Serving and Deployment
   - Simplifies serving with tools like KFServing for deploying and managing models.
   - Supports REST and gRPC interfaces for production-grade deployments.
7. Open-Source and Community-Driven
   - Backed by a strong open-source community with continuous updates and enhancements.
   - Supported by enterprise adopters and contributors for long-term growth.
8. Visualization and Monitoring
   - Provides interactive dashboards for pipeline monitoring and management.
   - Integrates with TensorBoard for model performance visualization.

KubeFlow Dashboard



Pros Of KubeFlow

- End-to-End ML Support:

  Handles the complete ML lifecycle, from data preparation to model deployment.

- Scalability:

  Leverages Kubernetes for scalable and distributed workflows.

- Framework Agnostic:

  Supports popular ML frameworks like TensorFlow, PyTorch, and Scikit-learn.

- Portability:

  Runs on any Kubernetes-supported infrastructure, including cloud and on-premises.

- Reproducibility:

  Ensures consistency with automatic experiment and artifact tracking.

- Customization:

  Offers flexibility to design custom components and workflows.

Cons Of KubeFlow

- Complex Setup:

  Initial installation and configuration can be challenging for beginners.

- Learning Curve:

  Requires knowledge of Kubernetes, which may be steep for data scientists without DevOps experience.

- Resource-Intensive:

  Demands significant infrastructure, which can be costly for small-scale projects.

- Overhead for Small Projects:

  Better suited for large, scalable ML workflows than smaller, simpler tasks.

Hardware Requirements

CPU

- Minimum: Dual-core processor (e.g., Intel Core i3 or AMD Ryzen 3)
- Suitable for small-scale deployments and light workloads.
- Recommended: Quad-core processor or higher (e.g., Intel Core i5/i7 or AMD Ryzen 5/7)
- Essential for running production-grade workloads and managing distributed tasks efficiently.

RAM

- Minimum: 4 GB
- Adequate for small-scale testing and handling simple workflows.
- Recommended: 16 GB or higher
- Necessary for processing large datasets, handling multiple pipelines, and ensuring smooth

Hardware Requirements

Storage

- Minimum: 50 GB of free disk space
- Required for storing datasets, Kubeflow metadata, intermediate results, and logs.
- Recommended: SSD with at least 256 GB capacity
- Provides faster I/O operations, crucial for performance in I/O-intensive workflows.

GPU (Optional, for ML Workflows)

- Recommended: A CUDA-enabled GPU (e.g., NVIDIA GTX 1650 or higher)
- Accelerates machine learning model training, large-scale data processing, and computationally intensive tasks.

Software Requirements

Operating System

- Minimum: Linux-based OS (e.g., Ubuntu 18.04 or CentOS 7)
- Required for compatibility with Kubernetes and containerized workloads.
- Recommended: Latest LTS version of a Linux-based OS (e.g., Ubuntu 22.04 LTS)
- Ensures long-term support, security updates, and improved performance.

Kubernetes

- Minimum: Kubernetes 1.21 or later
- Necessary for deploying and managing Kubeflow pipelines and components.
- Recommended: Kubernetes 1.25 or later
- Supports enhanced features, scalability, and compatibility with recent Kubeflow versions.

Container Runtime

- Minimum: Docker 19.03 or containerd 1.4
- Required for running containerized workloads in Kubernetes

Software Requirements

Python

- Minimum: Python 3.6
- Needed for running Kubeflow pipelines and Python-based machine learning tasks.
- Recommended: Python 3.8 or later
- Provides access to recent features and better library support.

Kubeflow CLI Tool (kubectl):

- Minimum: Version 1.21 (matching Kubernetes version)
- Recommended: Latest stable version
- Used for interacting with Kubernetes clusters.

Browser (for Kubeflow Dashboard)

- Minimum: Chromium-based browser (e.g., Chrome 89 or later) or Firefox 87 or later
- Required for accessing the Kubeflow UI.
- Recommended: Latest version of Google Chrome or Firefox

Dataset Management

Kubeflow demonstrates robust dataset management capabilities through several key features:

- Versioning and Tracking: Integration with tools like Data version control(DVC) or ML Metadata ensures reproducibility by tracking dataset versions and changes.
- Modular Pipelines: Clear separation of stages such as data loading, preprocessing, and model training using Kubeflow Pipelines.
- Extensible Data Format Support: Handles various data formats through Kubernetes-backed storage and Python libraries for flexibility.

Testing and Deployment Challenges

The implementation of Kubeflow reveals several testing and deployment challenges:

- Configuration of **multi-version pipelines** to handle different workflow requirements and updates.
- Artifact management for datasets, models, and metadata using Kubernetes persistent volumes or external storage systems.
- Version control and lineage tracking for models and pipeline runs to ensure traceability and reproducibility.
- Comprehensive error handling and logging mechanisms across distributed pipeline steps.
- Validation of pipeline inputs, such as data quality, schema checks, and resource specifications.
- Scalability considerations in distributed processing and model inference for high-demand scenarios.

Data Governance in Kubeflow

Data Validation and Quality Control

- Column Validation: Integration with tools like TensorFlow Data Validation (TFDV) for schema checks and anomaly detection, with clear error messages for missing or invalid columns.
- Missing Value Handling: Preprocessing steps for imputation or removal, with data cleaning steps tracked via metadata.
- Data Drift Detection: Monitoring for data drift using TFDV or custom scripts to trigger re-training.
- Data Provenance: ML Metadata (MLMD) tracks data lineage and transformations for reproducibility.
- Data Lineage: Kubeflow offers visualisation of data lineage of a particular artifacts

Data Governance in Kubeflow

Logging and Audit Trail

- Logging Configuration: Standardized logging with timestamps, hierarchical levels, and component identification via Kubernetes-native tools.
- Operation Tracking: Automated tracking of data loading, processing success/failure, and error conditions in pipelines.
- Centralized Monitoring: Integration with Prometheus and Grafana for real-time metrics and logs visualization.

Data Governance in Kubeflow

Additional Features

- End-to-End Metadata: Comprehensive tracking of artifacts and workflows using MLMD.
- Resource Optimization: Autoscaling through Kubernetes for efficient pipeline execution.
- Experimentation Support: Comparison and tracking of multiple pipeline runs for reproducibility.
- Model Monitoring: KFServing integration for model deployment and performance monitoring.

Some kubernates terms

Pod:A pod is **the smallest unit that can be deployed and managed**

Namespace: A namespace is a logical unit that organizes, manages, and secures resources within a cluster

Artifacts: Artifacts in Kubernetes are resources that are updated during a deployment or delivery pipeline.

THANK YOU!

FOR YOUR ATTENTION